

Part4 Assembly code generator

Start Assignment

- Due Tuesday by 11:59pm
- Points 20
- Submitting a website url
- Available until Mar 12 at 11:59pm

In this part of the project, you will build the backend of the compiler. The backend of a compiler has two parts to it: (a) Register allocation (b) Assembly code generation. Here are some tests to test your final compiler: [assembly_gen_tests.zip \(https://dartmouth.instructure.com/courses/76748/files/15742474?wrap=1\)](https://dartmouth.instructure.com/courses/76748/files/15742474?wrap=1) [↓ \(https://dartmouth.instructure.com/courses/76748/files/15742474/download?download_frd=1\)](https://dartmouth.instructure.com/courses/76748/files/15742474/download?download_frd=1)

Input: LLVM IR

Output: Assembly code for x86 32-bit architecture

Register Allocation

Overview

As discussed in class, we will perform local register allocation (at the basic block level). When performing local register allocation, all physical registers (read the special note below) are free at the entry point of each basic block. At the exit point of a basic block, all the variables that will be used in other basic blocks must be stored in memory. You do not have to do anything special for this, as LLVM, by default, creates the required store instructions at the end of each basic block.

Output: A map (**reg_map**) from **LLVMValueRef (a LLVM instruction)** to **Assigned physical register**. Note that the keys in this map are references to those instructions that generate a value (has a LHS in LLVM IR code). Your register allocation algorithm can use the same map to hold the information computed for each basic block. The map will also contain those values (instructions) that do not have a physical register assigned (spills). The value of assigned physical register for these will be -1 to indicate that they are spilled.

You will be implementing the [linear-scan register allocation algorithm](http://web.cs.ucla.edu/~palsberg/course/cs132/linearscan.pdf) [↗](http://web.cs.ucla.edu/~palsberg/course/cs132/linearscan.pdf) (<http://web.cs.ucla.edu/~palsberg/course/cs132/linearscan.pdf>) at basic block level. To implement this algorithm you will need to compute the liveness range of each variable in the basic block for which register allocation is being performed. You will define **compute_liveness** function that will compute the liveness range for each value in the basic block. This function takes the reference of a basic block and populates two data-structures:

1) a map (**inst_index**), that maps the instructions to their index in the basic-block (first instruction is at index 0). The **alloca** instructions are ignored while creating the indices.

2) a map (**live_range**), that maps the instructions in the given basic block to a pair of integers (start, end). The first integer in the pair indicates the index of the instruction that defines the value and second number gives the index of the last instruction that uses the value in the current basic block.

The information stored in these two data structures is local to each basic block, so make sure you clear them for each basic block.

Register allocation algorithm (Based on linear-scan algorithm):

Input:

1. Optimized LLVM IR code

Output: A map (**reg_map**) from **LLVMValueRef** (a LLVM instruction) to an **assigned physical register**. Note that the keys in this map are references to those instructions that generate a value (has a LHS in LLVM IR code). Your register allocation algorithm can use the same map to hold the information computed for each basic block. The map will also contain those values (instructions) that do not have a physical register assigned (spills). The value of assigned physical register for these will be -1 to indicate that they are spilled.

1. for each basic block **B** in the function
 1. Initialize the set of available physical registers to (ebx, ecx, edx)
 2. Get **inst_index** for basic block **B**
 3. Get **live_range** and by calling **compute_liveness** on the **B**
 4. for each instruction **Instr** in the basic block
 1. Ignore **Instr** if it is an alloc instruction
 2. If **Instr** is an instruction that does not have a result (e.g. store, branch, call that doesn't return a value):
 1. If live range of any operand of **Instr** ends, and it has a physical register **P** assigned to it then add **P** to available set of registers.
 3. If instruction **Instr** is not ignored above, then one of the three following cases may occur:
 1. If the following conditions about **Instr** hold: **a)** is of type add/sub/mul **b)** the *first* operand has a physical register **R** assigned to it in **reg_map** and **c)** liveness range of the *first* operand ends at **Instr**, then
 1. Add the entry **Instr** -> **R** to **reg_map** (Note: **Instr** will be a LLVMValueRef)
 2. If live range of *second* operand of **Instr** ends, and it has a physical register **P** assigned to it then add **P** to available set of registers.
 2. If a physical register **R** is available:
 1. Add the entry **Instr** -> **R** to **reg_map** (Note: **Instr** will be a LLVMValueRef)
 2. Remove **R** from the available pool of registers
 3. If live range of any operand of **Instr** ends, and it has a physical register **P** assigned to it then add **P** to available set of registers.
 3. If a physical register is not available:

1. Call **find_spill** to find the LLVMValueRef to spill based on heuristic, let this LLVMValueRef be **V**
2. If **V** has more uses than **Instr** (or liveness range of **Instr** ends after liveness range of **V**, note the choice is based on your heuristic to spill)
 1. Add the entry **Instr** -> -1 to **reg_map** to indicate that it does not have a physical register assigned
3. else:
 1. Get the physical register **R** assigned to **V** in **reg_map**
 2. Add the entry **Instr** -> **R** to **reg_map**
 3. Update the entry for **V** from **V** -> **R** to **V** -> -1 in **reg_map**
4. If the live range of any operand of **Instr** ends, and it has a physical register **P** assigned to it, then add **P** to the available set of registers.

find_spill helper function:

Input:

1. **Instr**: The instruction that is being assigned a register
2. **reg_map**
3. **inst_index**
4. sorted list **sorted_list** of LLVMValueRefs (instructions). There are two different heuristics by which the list can be sorted:
 1. By the endpoints in live_map (decreasing order)
 2. By number of uses of LLVMValueRefs (increasing order)
5. **live-range** map

Output: A LLVMValueRef, **V**, that has a physical register assigned to it

Pseudocode:

1. for every instruction **V** in **sorted_list** that has overlapping liveness with **Instr**:
 1. If **V** is in **reg_map** and the associated value is not -1:
 1. return **V**
2. return NULL (if no instructions match)

Special note:

Here is the list of assembly instructions that you will be using to generate assembly code: [Assembly instructions \(https://dartmouth.instructure.com/courses/76748/pages/assembly-instructions\)](https://dartmouth.instructure.com/courses/76748/pages/assembly-instructions)

Here are a few special cases that you need to note:

1. I have not included any operations for divide, as the divide is handled differently than all other arithmetic operations on x86. For this part of the project, we will assume that our minic program will not have divide operations.

2. For register allocation, we will assume only three physical registers are available for register allocation (**ebx**, **ecx** and **edx**). We will use **eax** as a special register for the following purposes:
 1. Returning a value
 2. Getting the returned value from a function call
 3. As a temporary register when the value being defined by add/sub/mul/cmp operations does not have an assigned physical register

Assembly code generation

Using the information computed by register allocation, you need to generate assembly code from LLVM IR. In the description given below *emit* means to write an assembly instruction to the output file (given as a command-line argument). Here is a reference [document to x86 instruction set](http://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html)  (<http://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html>).

You will be defining the following helper functions:

- 1) **createBBLabels**: This function populates a map where the key is an **LLVMBasicBlockRef** and the associated value is a char *, which you can use as a label when generating code.
- 2) **printDirectives**: This function emits the required directives for your function.
- 3) **printFunctionEnd**: This function emits the assembly instructions to restore the value of **%esp** and **%ebp** (you can do this by using the **leave** instruction instead of explicit moves), and the **ret** instruction.
- 4) **getOffsetMap**: This function will populate a map **offset_map**. This map associates each value(instruction) to the memory offset of that value from **%ebp**. The keys in this map are **LLVMValueRef**, and the values are integers. This function will also initialize an integer variable **localMem** that indicates the number of bytes required to store the local values.

getOffsetMap algorithm:

[Note: This assumes that every expression has only two operands, and the result is stored in a variable before using it in another expression]

Input: LLVM module

Output: A map (**offset_map**) from LLVMValueRef to an integer

1. Initialize **localMem** to 4.
2. If the function has a parameter
 - A. Get the LLVMValueRef of the parameter using LLVMGetParam. Add the LLVMValueRef of the parameter as key and its associated value as 8 to the **offset_map**.
3. For each basic block in the function
 1. For each instruction **instr** in the basic block
 - A. If **instr** is an alloc instruction

1. Add 4 to the **localMem**.
 2. Add **instr** as the key with the associated value as $(-1 * \text{localMem})$ to **offset_map**.
- B. If **instr** is a store instruction
1. If the first operand of the store instruction is equal to the function parameter
 - A. Get the value associated with the *first* operand in **offset_map**. Let this be **x**.
 - B. Change the value associated with the *second* operand in **offset_map** to **x**.
 2. If the first operand of the store instruction is not equal to the function parameter
 - A. Get the value associated with the *second* operand in **offset_map**. Let this be **x**.
 - B. Add the *first* operand as the key with the associated value as **x** in **offset_map**.
- C. If **instr** is a load instruction
1. Get the value associated with the *first* operand in **offset_map**. Let this be **x**.
 2. Add **instr** as the key with the associated value as **x** in the **offset_map**.

Code generation algorithm

1. for each function defined in your module (in a minic program there will be only one function):
 1. **createBBLabels**
 2. call **printDirectives** (.text, .globl, .type, function label)
 3. call **getOffsetMap** to initialize **offset_map** and **localMem**
 4. emit **pushl %ebp**
 5. emit **movl %esp, %ebp**
 6. emit **subl localMem, %esp**
 7. emit **pushl %ebx**
 8. for each basicBlock **BB** in the function:
 1. print the basic block label
 2. for each instruction **Instr** in **BB**:
 1. if **Instr** is a return: (**ret i32 A**)
 1. If **A** is a constant
 1. emit **movl \$A, %eax**
 2. If **A** is a temporary variable and is in memory
 1. get offset **k** of **A**
 2. emit **movl k(%ebp), %eax**
 3. If **A** is a temporary variable and has a physical register **%exx** assigned to it
 1. emit **movl %exx, %eax**
 4. emit **popl %ebx**
 5. call **printFunctionEnd** (leave, ret)
 2. if **Instr** is a load: (**%a = load i32, i32* %b**)
 1. if **%a** has been assigned a physical register **%exx**:
 1. get offset **c** for **%b**
 2. emit **movl c(%ebp), %exx**
 3. if **Instr** is a store: (**store i32 A, i32* %b**)

1. If **A** is a parameter
 1. ignore
2. If **A** is a constant
 1. get offset **c** of **%b**
 2. emit **movl \$A, c(%ebp)**
3. If **A** is a temporary variable **%a**
 1. if **%a** has a physical register **%exx** assigned to it:
 1. get offset **c** of **%b**
 2. emit **movl %exx, c(%ebp)**
 2. else:
 1. get offset **c1** of **%a**
 2. emit **movl c1(%ebp), %eax**
 3. get offset **c2** of **%b**
 4. emit **movl %eax, c2(%ebp)**
4. if **Instr** is a call: (**%a = call type @func()**) or (**call type @func(P)**)
 1. emit **pushl %ecx**
 2. emit **pushl %edx**
 3. if **func** takes a param **P**
 1. if **P** is constant
 1. emit **pushl \$P**
 2. if **P** is a temporary variable **%p**
 1. if **%p** has a physical register **%exx** assigned
 1. emit **pushl %exx**
 2. if **%p** is in memory
 1. get offset **k** of **%p**
 2. emit **pushl k(%ebp)**
 4. emit **call func**
 5. if **func** has a param **P**
 1. emit **addl \$4, %esp** (this is to undo the pushing of the parameter)
 6. emit **popl %edx**
 7. emit **popl %ecx**
 8. if **Instr** is of the form (**%a = call type @func()**)
 1. if **%a** has a physical register **%exx** assigned to it
 1. emit **movl %eax, %exx**
 2. if **%a** is in memory
 1. get offset **k** of **%a**
 2. emit **movl %eax, k(%ebp)**
 5. if **Instr** is a branch (**br i1 %a, label %b, label %c**) or (**br label %b**)
 1. if the branch is unconditional (**br label %b**)
 1. get label **L** of **%b** from **bb_labels**

2. emit **jmp L**
2. if the branch is conditional (**br i1 %a, label %b, label %c**)

[Note: %b is operand 2 and %c is operand 1 when you use LLVMGetOperand of branch instruction]

 1. get labels **L1** for %b and **L2** for %c from **bb_labels**
 2. get the predicate **T** of comparison from instruction %a (<, >, <=, >=, ==)
 3. based on value of **T**:
 1. emit **jxx L1** [replace jxx with conditional jump assembly instruction corresponding to **T**]
 4. emit **jmp L2**
6. if **Instr** is arithmetic (**add/mul/sub**) instruction: (**%a = add nsw A, B**)
 1. Let **R** be the register for %a. If %a has a physical register %**exx** assigned to it, then **R** is %**exx**, else **R** is %**eax**.
 2. if **A** is a constant
 1. emit **movl \$A, R**
 3. if **A** is a temporary variable
 1. if **A** has physical register %**eyy** assigned to it:
 1. emit **movl %eyy, R** (do not emit if both registers are the same)
 2. if **A** is in memory:
 1. get offset **n** of **A**
 2. emit **movl n(%ebp), R**
 4. if **B** is a constant
 1. emit **addl \$B, R** [Note: you can replace addl by subl or imull based on opcode]
 5. if **B** is a temporary variable and has physical register %**ezz** assigned to it:
 1. emit **addl %ezz, R** [Note: you can replace addl by subl or imull based on opcode]
 6. if **B** is a temporary variable and does not have a physical register assigned to it:
 1. get offset **m** of **B**
 2. emit **addl m(%ebp), R** [Note: you can replace addl by subl or imull based on opcode]
 7. if %a is in memory
 1. get offset **k** of %a
 2. emit **movl %eax, k(%ebp)**
7. if **Instr** is a compare instruction: (**%a = icmp slt A, B**)
 1. Let **R** be the register for %a. If %a has a physical register %**exx** assigned to it, then **R** is %**exx**, else **R** is %**eax**.
 2. if **A** is a constant
 1. emit **movl \$A, R**
 3. if **A** is a temporary variable
 1. if **A** has physical register %**eyy** assigned to it:

1. emit **movl %eyy, R** (do not emit if both registers are the same)
2. if **A** is in memory:
 1. get offset **n** of **A**
 2. emit **movl n(%ebp), R**
4. if **B** is a constant
 1. emit **cmpl \$B, R**
5. if **B** is a temporary variable and has physical register **%ezz** assigned to it:
 1. emit **cmpl %ezz, R**
6. if **B** is a temporary variable and does not have a physical register assigned to it:
 1. get offset **m** of **B**
 2. emit **cmpl m(%ebp), R**